LAPORAN

WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)

SEMESTER III



BKPM Acara Acara 13 Service Layer, Validasi, Session, dan Business Logic

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Pengembangan aplikasi web berbasis server membutuhkan pengelolaan kode program yang terstruktur agar aplikasi mudah dipahami, dikembangkan, dan dipelihara. Pada aplikasi yang memiliki fitur pengelolaan data, seperti sistem informasi akademik, proses penerimaan permintaan dari pengguna, validasi data, dan penyimpanan ke database perlu dipisahkan berdasarkan tanggung jawab masing-masing komponen.

Salah satu pendekatan yang dapat digunakan adalah pemisahan antara Controller, Service, dan Repository. Controller bertugas menerima permintaan pengguna dan menentukan respons yang diberikan. Service bertanggung jawab menangani logika bisnis serta validasi data, sedangkan Repository berfungsi mengelola komunikasi dengan database melalui operasi seperti menampilkan, menambahkan, mengubah, dan menghapus data.

Selain pemisahan tanggung jawab tersebut, aplikasi web juga membutuhkan mekanisme penyampaian informasi kepada pengguna. Salah satu mekanisme yang digunakan adalah flash message, yaitu pesan sementara yang disimpan menggunakan session dan ditampilkan setelah proses tertentu selesai. Pesan tersebut dapat berupa informasi keberhasilan, kegagalan, maupun kesalahan validasi.

Pada Acara 13 Workshop Web Server, dilakukan pengembangan aplikasi Sistem Informasi Akademik dengan menerapkan Service Layer, validasi, session, dan business logic. Implementasi dilakukan dengan menggunakan bahasa pemrograman PHP serta memanfaatkan struktur program yang telah tersedia pada project praktikum.

Melalui praktikum ini, diharapkan struktur kode menjadi lebih terorganisasi, proses validasi dapat dikelola secara terpusat, akses database dilakukan melalui Repository, dan pengguna memperoleh informasi yang sesuai setelah melakukan operasi terhadap data mahasiswa.

1.2 Rumusan Masalah

Rumusan masalah dalam praktikum ini adalah sebagai berikut.

1. Bagaimana menerapkan pola Service Layer pada aplikasi Sistem Informasi Akademik?
2. Bagaimana memisahkan validasi dan logika bisnis dari Controller?
3. Bagaimana mengelola operasi database melalui Repository?
4. Bagaimana menerapkan session untuk menampilkan flash message?
5. Bagaimana menangani proses penambahan, perubahan, dan penghapusan data mahasiswa?

BAB 2. LANDASAN TEORI

2.1 Aplikasi Web Berbasis PHP

PHP merupakan bahasa pemrograman yang banyak digunakan untuk mengembangkan aplikasi web pada sisi server. PHP dapat menerima data dari formulir, memproses permintaan pengguna, berkomunikasi dengan database, serta menghasilkan respons yang ditampilkan melalui browser.

Dalam praktikum ini, PHP digunakan untuk membangun aplikasi Sistem Informasi Akademik yang menyediakan fitur pengelolaan data mahasiswa.

2.2 Controller

Controller merupakan komponen yang bertugas menangani permintaan pengguna dan mengatur alur proses aplikasi. Controller menerima input, memanggil Service untuk menjalankan proses bisnis, kemudian menentukan respons berupa tampilan atau pengalihan halaman.

Controller sebaiknya tidak berisi seluruh proses validasi dan query SQL secara langsung. Pemisahan tersebut membuat kode Controller lebih sederhana dan mudah dipahami.

2.3 Service Layer

Service Layer merupakan lapisan yang menampung logika bisnis aplikasi. Pada praktikum ini, `MahasiswaService` bertanggung jawab melakukan normalisasi input, validasi NIM, validasi nama, pemeriksaan program studi, validasi status mahasiswa, serta menentukan hasil proses tambah, ubah, dan hapus data.

Service menerima input dari Controller, menjalankan aturan bisnis, lalu memanggil Repository apabila data memenuhi ketentuan yang ditetapkan.

2.4 Repository Pattern

Repository Pattern merupakan pola desain yang memisahkan operasi pengaksesan data dari bagian lain aplikasi. Repository menjadi penghubung antara Service dan database.

Pada aplikasi yang dikerjakan, `MahasiswaRepository` menangani operasi untuk menampilkan, mencari, menambahkan, mengubah, dan menghapus data mahasiswa. Dengan demikian, query SQL tidak perlu dituliskan secara langsung di dalam Controller.

2.5 Validasi Data

Validasi data adalah proses pemeriksaan input sebelum data diproses atau disimpan ke database. Validasi diperlukan agar data yang masuk sesuai dengan aturan aplikasi.

Contoh validasi pada data mahasiswa meliputi:

1. NIM wajib diisi.
2. NIM hanya boleh berisi huruf dan angka.
3. Panjang NIM maksimal 20 karakter.
4. NIM tidak boleh sama dengan NIM mahasiswa lain.
5. Nama wajib diisi dan maksimal 100 karakter.
6. Program studi harus tersedia.
7. Status mahasiswa harus berupa `aktif`, `lulus`, atau `cuti`.

2.6 Session

Session adalah mekanisme untuk menyimpan informasi sementara di sisi server selama sesi pengguna berlangsung. Dalam PHP, session dapat digunakan untuk menyimpan pesan keberhasilan, pesan kesalahan, serta input formulir yang perlu ditampilkan kembali setelah proses pengalihan halaman.

Session pada aplikasi diinisialisasi menggunakan fungsi `session_start()`.

2.7 Flash Message

Flash message adalah pesan sementara yang digunakan untuk memberikan informasi kepada pengguna setelah suatu proses dilakukan. Pesan ini biasanya ditampilkan satu kali, kemudian dihapus dari session.

Contoh flash message dalam aplikasi adalah:

a. Data mahasiswa berhasil ditambahkan.
b. Data mahasiswa berhasil diubah.
c. Data mahasiswa berhasil dihapus.
d. NIM sudah terdaftar.

e. Data gagal disimpan.

2.8 PDO

PDO (PHP Data Objects) merupakan antarmuka PHP untuk mengakses database. PDO mendukung penggunaan prepared statement sehingga parameter query dapat dikirim secara terpisah dari perintah SQL.

Pada praktikum ini, Repository menggunakan PDO untuk menjalankan query database, misalnya ketika menampilkan daftar mahasiswa atau menyimpan data mahasiswa baru.

2.9 Arsitektur Controller, Service, dan Repository

Alur pengolahan data pada aplikasi dapat digambarkan sebagai berikut:

Pengguna → Controller → Service → Repository → Database

Setelah operasi selesai, hasilnya dikembalikan melalui Service kepada Controller. Controller kemudian menentukan respons dan, jika diperlukan, menyimpan flash message melalui session sebelum mengalihkan pengguna ke halaman berikutnya.

Pemisahan tersebut membuat setiap komponen memiliki tanggung jawab yang jelas dan mengurangi ketergantungan langsung antarkomponen.

BAB 3. PELAKSANAAN PRAKTIKUM

3.1 Hasil Praktikum

A. Pembuatan kode MahasiswaService.php

```php
<?php

require_once __DIR__ . '/../Models/MahasiswaRepository.php';
require_once __DIR__ . '/../Models/ProdiRepository.php';

class MahasiswaService
{
    private const STATUS_VALID = ['aktif', 'lulus', 'cuti'];

    public function __construct(
        private MahasiswaRepository $repo,
        private ProdiRepository $prodiRepo
    ) {}

    public function all(): array
    {
        return $this->repo->all();
    }

    public function find(int $id)
    {
        return $this->repo->find($id);
    }

    public function findDetail(int $id)
    {
        return $this->repo->findDetail($id);
    }

    public function prodiOptions(): array
    {
        return $this->prodiRepo->names();
    }

    public function create(array $input): array
    {
        $input  = $this->normalize($input);
        $errors = $this->validate($input);

        if (!empty($errors)) {
```

```
            return $this->fail($errors);
        }

        try {
            $id = $this->repo->create($input);
        } catch (PDOException $e) {
            return $this->fail($this->databaseErrors($e));
        }

        return [
            'success' => true,
            'id'      => $id,
            'message' => 'Data mahasiswa berhasil ditambahkan',
        ];
    }


    public function update(int $id, array $input): array
    {
        if ($id <= 0 || !$this->repo->find($id)) {
            return $this->fail(['_global' => 'Data mahasiswa
tidak ditemukan']);
        }

        $input  = $this->normalize($input);
        $errors = $this->validate($input, $id);

        if (!empty($errors)) {
            return $this->fail($errors);
        }

        try {
            $this->repo->update($id, $input);
        } catch (PDOException $e) {
            return $this->fail($this->databaseErrors($e));
        }

        return [
            'success' => true,
            'id'      => $id,
            'message' => 'Data mahasiswa berhasil diubah',
        ];
    }

    public function delete(int $id): array
    {
        if ($id <= 0 || !$this->repo->find($id)) {
```

```
            return $this->fail(['_global' => 'Data mahasiswa
tidak ditemukan']);
        }

        try {
            $this->repo->delete($id);
        } catch (PDOException $e) {
            return [
                'success' => false,
                'errors'  => ['_global' => 'Data gagal
dihapus'],
            ];
        }

        return [
            'success' => true,
            'message' => 'Data mahasiswa berhasil dihapus',
        ];
    }

    private function validate(array $input, ?int $exceptId =
null): array
    {
        $errors = [];

        // NIM
        if ($input['nim'] === '') {
            $errors['nim'] = 'NIM wajib diisi';
        } elseif (!preg_match('/^[A-Za-z0-9]+$/',
$input['nim'])) {
            $errors['nim'] = 'NIM hanya boleh berisi huruf dan
angka';
        } elseif (strlen($input['nim']) > 20) {
            $errors['nim'] = 'NIM maksimal 20 karakter';
        } elseif ($this->repo->existsByNim($input['nim'],
$exceptId)) {
            $errors['nim'] = 'NIM sudah terdaftar';
        }

        // Nama
        if ($input['nama'] === '') {
            $errors['nama'] = 'Nama wajib diisi';
        } elseif (strlen($input['nama']) > 100) {
            $errors['nama'] = 'Nama maksimal 100 karakter';
        }

        // Prodi
```

```
        if ($input['prodi'] === '') {
            $errors['prodi'] = 'Program studi wajib dipilih';
        } elseif
(!$this->prodiRepo->existsByNama($input['prodi'])) {
            $errors['prodi'] = 'Program studi tidak tersedia';
        }

        // Status
        if ($input['status'] === '') {
            $errors['status'] = 'Status wajib dipilih';
        } elseif (!in_array(strtolower($input['status']),
self::STATUS_VALID, true)) {
            $errors['status'] = 'Status tidak valid';
        }

        return $errors;
    }

    // Bersihkan spasi dan pastikan semua key ada (tidak ada
undefined index)
    private function normalize(array $input): array
    {
        return [
            'nim'    => trim((string) ($input['nim'] ?? '')),
            'nama'   => trim((string) ($input['nama'] ?? '')),
            'prodi'  => trim((string) ($input['prodi'] ?? '')),
            'status' => trim((string) ($input['status'] ??
'')),
        ];
    }

    // Terjemahkan error database menjadi pesan untuk pengguna
    private function databaseErrors(PDOException $e): array
    {
        // 23000 = pelanggaran constraint (mis. NIM duplikat
yang lolos
        // dari pengecekan karena ada request lain di waktu
bersamaan)
        if ($e->getCode() === '23000') {
            return ['nim' => 'NIM sudah terdaftar'];
        }

        return ['_global' => 'Data gagal disimpan'];
    }

    private function fail(array $errors): array
    {
```

```
        return [
            'success' => false,
            'errors'  => $errors,
        ];
    }
}
```

B. Kode MahasiswaController.php bagian yang menerima data dan memanggil service.

```
   public function store()
    {
        AuthMiddleware::handle();
        $this->mustBePost();

        $result = $this->service->create($_POST);

        $this->respond($result, $_POST, BASE_URL .
'/mahasiswa/create');
    }
   public function update()
    {
        AuthMiddleware::handle();
        $this->mustBePost();

        $id = (int) ($_POST['id'] ?? 0);

        $result = $this->service->update($id, $_POST);

        $this->respond($result, $_POST, BASE_URL .
'/mahasiswa/edit?id=' . $id);
    }
```

C. Kode MahasiswaRepository.php mengambil data mahasiswa berdasakan nim, fungsi ini dapat membantu apakah nim sudah di dipakai/digunakan.

```
  public function existsByNim($nim, $exceptId = null)
   {
       $sql = "SELECT COUNT(*) FROM mahasiswa WHERE nim = :nim";
       $params = [':nim' => $nim];

       if ($exceptId !== null) {
           $sql .= " AND id <> :id";
           $params[':id'] = $exceptId;
       }
```

```
        $stmt = $this->db->prepare($sql);
        $stmt->execute($params);

        return (int) $stmt->fetchColumn() > 0;
    }
```

D. Kode mahasiswa/index.php tampilan pesan flash

```
<h1>Data Mahasiswa</h1>

 <?php require __DIR__ . '/../partials/flash.php'; ?>
```

E. Hasil Praktikum

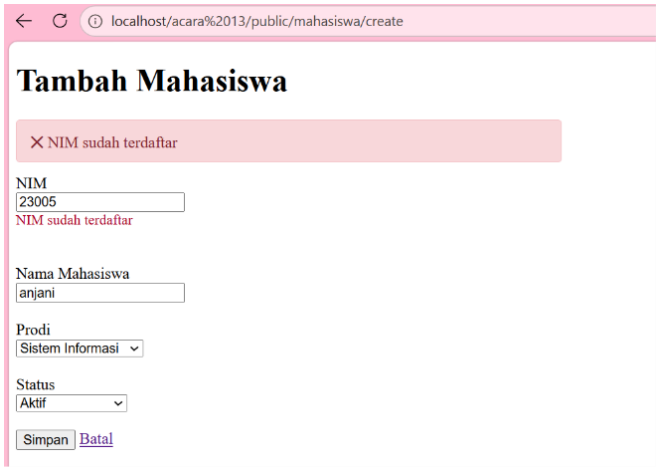
a. Pengujian tambah data mahasiswa



b. Pengujian NIM berupa angka



c. Pengujian jika NIM sama



d. Pengujian data mahasiswa di perbarui




localhost/acara%2013/public/mahasiswa/edit?id=1
Edit Mahasiswa
NIM
23001
Nama Mahasiswa
Azira Mutia Gani
Prodi
Teknik Informatika
Status
Lulus
Simpan Perubahan
Batal


localhost/acara%2013/public/mahasiswa
Data Mahasiswa
✓ Data mahasiswa berhasil diubah
+ Tambah Mahasiswa | Dashboard
ID NIM Nama Prodi Status Aksi
1 23001 Azira Mutia Gani Teknik Informatika lulus Edit Hapus
2 23002 Arshinta Puja Aulia Teknik Informatika cuti Edit Hapus
3 23003 Nindia Tri Anggraini Teknik Informatika lulus Edit Hapus
4 23004 Shofi Maulana Shofa Teknik Informatika aktif Edit Hapus
13 23005 anya Teknik Informatika lulus Edit Hapus

BAB 4. PEMBAHASAN

4.1 Pembahasan

Penerapan Service Layer, Validasi, Session, dan Business Logic pada praktikum Acara 13 bertujuan untuk memisahkan proses pengolahan data mahasiswa agar kode program lebih terstruktur. Pada praktikum ini, MahasiswaController.php digunakan untuk menerima permintaan dari pengguna dan memanggil MahasiswaService.php untuk menjalankan proses penambahan maupun perubahan data mahasiswa.

MahasiswaService.php berperan dalam menjalankan validasi dan business logic sebelum data diproses lebih lanjut. Sementara itu, MahasiswaRepository.php digunakan untuk mengelola akses database, termasuk memeriksa apakah NIM sudah terdaftar melalui fungsi existsByNim(). Pemeriksaan tersebut membantu mencegah penggunaan NIM yang sama oleh mahasiswa lain.

Selain itu, penggunaan Session melalui mekanisme Flash Message membantu menampilkan pesan kepada pengguna setelah proses pengolahan data dilakukan. Pada file mahasiswa/index.php, file flash.php dipanggil untuk menampilkan pesan keberhasilan maupun kesalahan. Dengan demikian, pengguna dapat mengetahui hasil proses yang dilakukan pada aplikasi.

Berdasarkan penerapan tersebut, pemisahan Controller, Service, dan Repository membantu membuat kode program lebih mudah dipahami dan dikelola. Controller berfokus pada alur permintaan dan respons, Service menangani validasi serta business logic, sedangkan Repository bertanggung jawab terhadap akses database. Mekanisme Flash Message juga membantu memberikan informasi kepada pengguna setelah proses pengolahan data.

BAB 5. PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum Acara 13, dapat disimpulkan bahwa penerapan Service Layer, Validasi, Session, dan Business Logic membantu meningkatkan keteraturan struktur aplikasi web. Pemisahan tugas antara Controller, Service, dan Repository membuat proses pengolahan data mahasiswa lebih terorganisasi.

Pada praktikum ini, MahasiswaController.php digunakan untuk menerima data dan memanggil Service, sedangkan MahasiswaRepository.php menyediakan fungsi untuk memeriksa penggunaan NIM pada database. Selain itu, mekanisme Flash Message digunakan untuk menampilkan pesan keberhasilan atau kesalahan kepada pengguna.

Dengan penerapan tersebut, pengembangan Sistem Informasi Akademik menjadi lebih terstruktur dan memudahkan pemeliharaan kode program.